Name: Salma Rani
Sap ID:54194
Course: Compiler Construction
Program:BSCS sem6
Submitted to:Mam Hasnaat

(Lab 7)

Task 1:

```
#include <iostream>
#include <string>
using namespace std;

string input;
int indexPos = 0;

void S();

void match(char expected)
{
    if(input[indexPos] == expected)
    {
        indexPos++;
    }
    else
    {
        cout << "Syntax Error\n";
        exit(0);
    }
}

// S → a S b | ε
void S()
{
    if(input[indexPos] == 'a')
    {
        match('a');   // match 'a'
        S();          // recursive call
        match('b');   // match 'b'
    }
    // else ε (do nothing)
}

int main()
{
    cout << "Enter string: ";
    cin >> input;

    S();
```

```
    if(indexPos == input.length())
        cout << "String Accepted\n";
    else
        cout << "String Rejected\n";

    return 0;
}
```

Output1:

```
Enter string: ab
String Accepted

--------------------------------
Process exited after 1.649 seconds with return value 0
Press any key to continue . . .
```

```
Enter string: aabb
String Accepted

--------------------------------
Process exited after 3.726 seconds with return value 0
Press any key to continue . . .
```

```
Enter string: aaabbb
String Accepted

--------------------------------
Process exited after 3.711 seconds with return value 0
Press any key to continue . . .
```

```
Enter string: ababab
String Rejected

--------------------------------
Process exited after 11.75 seconds with return value 0
Press any key to continue . . .
```

Explain how recursion matches the symbols.?

This parser works like a **stack (LIFO)**:

Key Idea:

Every time `'a'` is found → go deeper (recursive call)

Every recursive return → match one `'b'`

Step	Action
1	match a → go deeper
2	match a → go deeper
3	match a → go deeper
4	no 'a' → ε return
5	match b
6	return → match b
7	return → match b

So:

3 `'a'` → 3 recursive calls

3 `'b'` → matched during return

Task 2:

Step 1: Remove Left Recursion from E

Rule:
If
$A \to A\alpha \mid \beta$
Then convert to:

$A \to \beta A'$
$A' \to \alpha A' \mid \varepsilon$

Apply on E:

$E \to T E'$
$E' \to - T E' \mid \varepsilon$

Step 2: Remove Left Recursion from T

T → F T'
T' → * F T' | ε

Final Non-Left Recursive Grammar

E → T E'
E' → - T E' | ε
T → F T'
T' → * F T' | ε
F → i | €

Code:-

```
#include <iostream>
#include <string>
using namespace std;

string input;
int indexPos = 0;

void E();
void Eprime();
void T();
void Tprime();
void F();

void match(char expected)
{
   if(input[indexPos] == expected)
   {
      indexPos++;
   }
   else
   {
      cout << "Syntax Error\n";
      exit(0);
   }
}

// E → T E'
void E()
{
```

```
    T();
    Eprime();
}

// E' → - T E' | ε
void Eprime()
{
    if(input[indexPos] == '-')
    {
        match('-');
        T();
        Eprime();
    }
}

// T → F T'
void T()
{
    F();
    Tprime();
}

// T' → * F T' | ε
void Tprime()
{
    if(input[indexPos] == '*')
    {
        match('*');
        F();
        Tprime();
    }
}

// F → i | (E)
void F()
{
    if(input[indexPos] == 'i')
    {
        match('i');
    }
    else if(input[indexPos] == '(')
    {
        match('(');
```

```
        E();
        match(')');
    }
    else
    {
        cout << "Syntax Error\n";
        exit(0);
    }
}

int main()
{
    cout << "Enter expression: ";
    cin >> input;

    E();

    if(indexPos == input.length())
        cout << "String Accepted\n";
    else
        cout << "String Rejected\n";

    return 0;
}
```

Output2:

```
Enter expression: id-id*id
String Rejected

--------------------------------
Process exited after 38.81 seconds with return value 0
Press any key to continue . . .
```

```
Enter expression: i-i*i
String Accepted

--------------------------------
Process exited after 40.28 seconds with return value 0
Press any key to continue . . .
```

```
Enter expression: b-b*b
Syntax Error

--------------------------------
Process exited after 8.646 seconds with return value 0
Press any key to continue . . .
```

```
Enter expression: i*i+i-i
String Rejected

--------------------------------
Process exited after 11.36 seconds with return value 0
Press any key to continue . . .
```

Task 3:

```
#include <iostream>
#include <string>
using namespace std;

string input;
int i = 0;

void E();
void Eprime();
void T();
void Tprime();
void F();

void error() {
    cout << "Syntax Error\n";
    exit(0);
}

// E --> T E'
void E() {
    T();
    Eprime();
}

// E' --> + T E' | - T E' | e
```

```
void Eprime() {
    if (input[i] == '+') {
        i++;
        T();
        Eprime();
    }
    else if (input[i] == '-') {
        i++;
        T();
        Eprime();
    }
}

// T --> F T'
void T() {
    F();
    Tprime();
}

// T' --> * F T' | / F T' | e
void Tprime() {
    if (input[i] == '*') {
        i++;
        F();
        Tprime();
    }
    else if (input[i] == '/') {
        i++;
        F();
        Tprime();
    }
}

// F --> id | (E)
void F() {
    if (input.substr(i, 2) == "id") {
        i += 2;
    }
    else if (input[i] == '(') {
        i++;
        E();
        if (input[i] == ')')
            i++;
```

```
        else
            error();
    }
    else {
        error();
    }
}

int main() {
    cout << "Enter expression: ";
    cin >> input;

    E();

    if (i == input.length())
        cout << "String Accepted\n";
    else
        cout << "String Rejected\n";

    return 0;
}
```

Output 3:

```
Enter expression: id+id
String Accepted

--------------------------------
Process exited after 5.163 seconds with return value 0
Press any key to continue . . .
```

```
Enter expression: (id+id)*id
String Accepted

--------------------------------
Process exited after 26.8 seconds with return value 0
Press any key to continue . . .
```

```
Enter expression: id*id+id
String Accepted

--------------------------------
Process exited after 12.18 seconds with return value 0
Press any key to continue . . .
```

```
Enter expression: id+id/id-id
String Accepted

--------------------------------
Process exited after 8.211 seconds with return value 0
Press any key to continue . . .
```

```
Enter expression: id(id-id)/id(id+id)
String Rejected

--------------------------------
Process exited after 29.37 seconds with return value 0
Press any key to continue . . .
```

How Your Parser Works:-

E() → handles **+ and -**

T() → handles *** and /**

F() → handles **id and (E)**

Variable i → tracks current position in input

This ensures **operator precedence**:

*, / > +, -

Test 1: id + id

Step 1:

E() → T E'

Step 2: T()

T → F T'
F → id match "id"
T' → ε

pointer moves to +

Step 3: E'()

E' → + T E'
match '+'

Step 4: T()

T → F T'
F → id match "id"
T' → ε

Step 5:

E' → ε

Output: String Accepted

Test 2: (id + id) * id

Step 1:

E → T E'

Step 2: T()

T → F T'
F → (E)
match '('

Step 3: Inner E()

E → T E'
T → F T'
F → id

Step 4:

E' → + T E'
match '+'

T → F → id
E' → ε

Step 5:

match ')'

Step 6: T'()

T' → * F T'
match '*'
F → id
T' → ε

Step 7:

E' → ε

Output: String Accepted

Test 3: id * id + id

Step 1:

E → T E'

Step 2: T()

T → F T'
F → id

Step 3: T'()

T' → * F T'
match '*'
F → id
T' → ε

multiplication handled first

Step 4: E'()

E' → + T E'
match '+'

Step 5: T()

$T \rightarrow F\ T'$
$F \rightarrow id$ ✓
$T' \rightarrow \varepsilon$

Step 6:

$E' \rightarrow \varepsilon$

Output: String Accepted